

VoiceLead CRM — AI Voice Agent for Lead Qualification and Human Handoff

1. Project Overview

Build a production-style demo project called **VoiceLead CRM**.

VoiceLead CRM is an AI-powered voice agent and CRM system for local businesses. The system answers inbound calls, talks naturally with customers, collects lead information, qualifies the caller, calculates a lead score, saves the lead in a CRM dashboard, and alerts a real human team member when the lead is hot.

This project is designed as an Upwork portfolio demo for AI voice agents, lead automation, CRM workflows, and human handoff.

The goal is not to build a generic chatbot. The goal is to build a real business automation system that solves this problem:

Businesses miss calls, lose leads, and waste time manually qualifying people. This system answers calls automatically, identifies serious buyers, and sends hot leads to the right person immediately.

2. Demo Business

Use a fake local business for the demo:

Business Name: EliteFix Plumbing & HVAC

Industry: Home services

Services: AC repair, plumbing repair, water heater repair, emergency maintenance, installation requests

Location: Lahore, Pakistan

Working Hours: 9:00 AM – 8:00 PM

Emergency Support: Available for urgent AC/plumbing issues

Human Handoff Person: Sales/dispatch team member

The demo should show a caller contacting the company for an urgent AC repair. The AI agent should collect the details, score the lead as hot, save it in the CRM, and send a hot-lead alert.

3. Main User Flow

Customer Call Flow

1. Customer calls the business number.

2. AI voice agent answers:

“Hi, thanks for calling EliteFix Plumbing & HVAC. How can I help you today?”

3. Customer explains the issue.

4. AI asks qualification questions:

5. What service do you need?

6. Is this urgent?

7. When do you need help?

8. What area are you located in?

9. What is your name?

10. What is the best phone number for follow-up?

11. Are you ready to book or just checking prices?

12. AI summarizes the caller's need.

13. AI assigns a lead score.

14. Lead is saved in CRM.

15. If score is high, the system alerts a real person.

16. CRM dashboard shows transcript, summary, score, status, and next action.

4. Core Features

4.1 AI Voice Agent

The voice agent should:

- Answer inbound calls.
- Speak naturally.
- Ask qualification questions.
- Detect customer intent.
- Collect customer details.
- Identify urgency.
- Identify requested service.
- Capture preferred callback time.
- Summarize the call.
- Trigger webhook after the call.
- Escalate or mark hot leads for human follow-up.

For MVP, the actual phone layer can be mocked through a webhook/API form if voice integration is not available yet. But structure the project so Vapi, Retell, Twilio, or another voice provider can be integrated later.

4.2 Lead CRM Dashboard

Build a CRM dashboard where the business owner/team can see all leads.

Dashboard should include:

- Total leads
 - Hot leads
 - Warm leads
 - Cold leads
 - New leads today
 - Conversion status
 - Lead table
 - Lead detail page
 - Transcript view
 - AI summary
 - Lead score breakdown
 - Recommended next action
 - Status update buttons
-

4.3 Lead Scoring

Each lead should be scored from **0 to 100**.

Use this scoring model:

Criteria	Points
Clear service need detected	+20
Urgent timeline	+20
Customer provided phone number	+15
Customer provided name	+10
Customer is ready to book/callback	+15
Location is within service area	+10
Budget or seriousness signal detected	+10

Lead Categories

- **Hot Lead:** 75–100
- **Warm Lead:** 45–74
- **Cold Lead:** 0–44

Example Hot Lead

Caller says:

"My AC stopped working. Can someone come today? I am in DHA Lahore. My name is Ahmad and my number is 0300-1234567."

Score:

- Clear service need: +20
- Urgent timeline: +20
- Phone number: +15
- Name: +10
- Ready for callback/booking: +15
- In service area: +10
- Seriousness signal: +10

Total: **100/100**

Status: **Hot Lead**

Recommended action:

Call immediately and dispatch technician.

5. Human Handoff Logic

If lead score is **75 or higher**, trigger hot lead alert.

Hot lead alert should include:

- Lead name
- Phone number
- Service requested
- Urgency
- Location
- Lead score
- AI summary
- Recommended next action
- CRM lead link

Example alert:

 Hot Lead Alert

Name: Ahmad
Phone: 0300-1234567
Service: AC repair
Urgency: Today
Location: DHA Lahore
Score: 92/100

Summary:

Caller needs urgent AC repair today in DHA Lahore. They are ready for a callback and provided contact details.

Recommended Action:

Call immediately and schedule technician visit.

For MVP, send the alert by email or show it in the dashboard. Later integrations can include Slack, WhatsApp, SMS, or CRM webhook.

6. Recommended Tech Stack

Use the following stack:

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Hook Form
- Lucide icons

Backend

- Next.js API routes or server actions
- TypeScript
- Zod for validation

Database

Use Supabase PostgreSQL.

Auth

For MVP:

- Simple demo login
- Or no login initially

- Later add Supabase Auth

AI

Use OpenAI API or another LLM provider.

AI tasks:

- Extract lead data from transcript
- Summarize call
- Score lead
- Suggest next action

Voice Provider

Prepare integration for one of these:

- Vapi
- Retell
- Twilio Voice
- Mock webhook for MVP

Start with a mock voice webhook first. Then real voice can be added later.

7. App Pages

7.1 Dashboard Page

Route:

/

Show:

- Total leads
- Hot leads
- Warm leads
- Cold leads
- New leads today
- Recent hot leads
- Latest call activity

Cards:

Total Leads: 42
Hot Leads: 12
Warm Leads: 19
Cold Leads: 11
Today's Calls: 8
Avg Lead Score: 67

7.2 Leads Page

Route:

/leads

Show a table:

Name	Phone	Service	Location	Score	Status	Created	Action
Ahmad	0300-1234567	AC repair	DHA Lahore	92	Hot	Today	View
Sara	0312-2223333	Plumbing	Gulberg	63	Warm	Today	View
Ali	0321-8889999	Price inquiry	Unknown	30	Cold	Yesterday	View

Filters:

- All
- Hot
- Warm
- Cold
- New
- Contacted
- Won
- Lost

Search by:

- Name
- Phone
- Service
- Location

7.3 Lead Detail Page

Route:

/leads/[id]

Show:

- Caller name
- Phone number
- Email if available
- Service requested
- Location
- Urgency
- Lead score
- Lead status
- Call transcript
- AI summary
- Score breakdown
- Recommended next action
- Created date/time
- Last updated date/time

Buttons:

- Mark as Contacted
 - Mark as Won
 - Mark as Lost
 - Send Follow-up
 - Call Lead
 - Delete Lead
-

7.4 Settings Page

Route:

/settings

Business owner can edit:

- Business name
- Business phone
- Industry
- Services

- Service areas
 - Working hours
 - Handoff email
 - Handoff phone
 - Hot lead threshold
 - Warm lead threshold
 - AI greeting
 - AI fallback message
-

7.5 Demo Call Simulator Page

Route:

/demo-call

This page is very important for portfolio demo.

Build a form where I can paste a fake call transcript and click **Analyze Call**.

Example transcript:

AI: Hi, thanks for calling EliteFix Plumbing & HVAC. How can I help you today?
Caller: My AC stopped working and I need someone today.
AI: Sorry to hear that. What area are you located in?
Caller: DHA Lahore.
AI: Got it. May I have your name?
Caller: Ahmad.
AI: And what is the best phone number for follow-up?
Caller: 0300-1234567.
AI: Are you ready to book a technician visit today?
Caller: Yes, please send someone as soon as possible.

After clicking Analyze Call:

- AI extracts data
- AI calculates score
- Lead is saved
- Hot lead alert is generated
- User is redirected to lead detail page

This allows the project to work even before real voice integration.

8. Database Schema

Create these tables.

8.1 businesses

```
create table businesses (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  industry text,  
  phone text,  
  email text,  
  location text,  
  working_hours text,  
  services text[],  
  service_areas text[],  
  handoff_email text,  
  handoff_phone text,  
  hot_lead_threshold int default 75,  
  warm_lead_threshold int default 45,  
  ai_greeting text,  
  ai_fallback_message text,  
  created_at timestamp with time zone default now(),  
  updated_at timestamp with time zone default now()  
);
```

8.2 leads

```
create table leads (  
  id uuid primary key default gen_random_uuid(),  
  business_id uuid references businesses(id) on delete cascade,  
  name text,  
  phone text,  
  email text,  
  service_requested text,  
  location text,  
  urgency text,  
  budget text,  
  intent text,  
  lead_score int default 0,  
  lead_temperature text check (lead_temperature in ('hot', 'warm', 'cold'))  
  default 'cold',  
  status text check (status in ('new', 'contacted', 'won', 'lost',  
  'follow_up')) default 'new',
```

```
source text default 'voice_agent',
ai_summary text,
recommended_action text,
transcript text,
call_recording_url text,
score_breakdown jsonb,
raw_ai_output jsonb,
created_at timestamp with time zone default now(),
updated_at timestamp with time zone default now()
);
```

8.3 lead_events

```
create table lead_events (
  id uuid primary key default gen_random_uuid(),
  lead_id uuid references leads(id) on delete cascade,
  event_type text not null,
  description text,
  metadata jsonb,
  created_at timestamp with time zone default now()
);
```

Example event types:

```
lead_created
lead_scored
hot_lead_alert_sent
status_changed
follow_up_sent
human_handoff_triggered
```

8.4 notifications

```
create table notifications (
  id uuid primary key default gen_random_uuid(),
  lead_id uuid references leads(id) on delete cascade,
  channel text,
  recipient text,
  subject text,
  message text,
  status text default 'pending',
```

```
    sent_at timestamp with time zone,  
    created_at timestamp with time zone default now()  
);
```

9. API Routes

Create these API routes.

9.1 Create Lead Manually

```
POST /api/leads
```

Request body:

```
{  
  "name": "Ahmad",  
  "phone": "0300-1234567",  
  "service_requested": "AC repair",  
  "location": "DHA Lahore",  
  "urgency": "today",  
  "transcript": "Full call transcript here"  
}
```

Response:

```
{  
  "success": true,  
  "lead": {}  
}
```

9.2 Analyze Call Transcript

```
POST /api/ai/analyze-call
```

Request body:

```
{
  "transcript": "AI: Hi... Caller: My AC stopped working..."
}
```

Expected response:

```
{
  "name": "Ahmad",
  "phone": "0300-1234567",
  "email": null,
  "service_requested": "AC repair",
  "location": "DHA Lahore",
  "urgency": "today",
  "budget": null,
  "intent": "ready_to_book",
  "lead_score": 92,
  "lead_temperature": "hot",
  "ai_summary": "Caller needs urgent AC repair today in DHA Lahore and is ready
for a technician visit.",
  "recommended_action": "Call immediately and dispatch technician.",
  "score_breakdown": {
    "clear_service_need": 20,
    "urgent_timeline": 20,
    "phone_number_provided": 15,
    "name_provided": 10,
    "ready_to_book": 15,
    "in_service_area": 10,
    "seriousness_signal": 10
  }
}
```

9.3 Voice Provider Webhook

```
POST /api/voice/webhook
```

This endpoint receives call data from Vapi, Retell, Twilio, or mock test.

Request body example:

```
{
  "call_id": "call_123",
  "caller_phone": "0300-1234567",
}
```

```
"transcript": "Full call transcript here",
"recording_url": "https://example.com/recording.mp3",
"provider": "mock"
}
```

Flow:

1. Receive transcript.
2. Send transcript to AI analysis function.
3. Save lead to database.
4. If score $\geq$ hot threshold, trigger hot lead alert.
5. Return success.

Response:

```
{
  "success": true,
  "lead_id": "uuid",
  "lead_temperature": "hot",
  "lead_score": 92,
  "handoff_triggered": true
}
```

9.4 Send Hot Lead Alert

```
POST /api/notifications/hot-lead
```

Request body:

```
{
  "lead_id": "uuid"
}
```

Action:

- Fetch lead.
- Format alert message.
- Send email or save notification.
- Mark notification as sent.
- Create lead event.

For MVP, it is acceptable to save notification in database and show it in dashboard. Email integration can be added after.

9.5 Update Lead Status

```
PATCH /api/leads/[id]/status
```

Request body:

```
{  
  "status": "contacted"  
}
```

Allowed statuses:

```
new  
contacted  
won  
lost  
follow_up
```

10. AI Prompt for Call Analysis

Create a server-side AI function that sends the transcript to the LLM.

System prompt:

```
You are an AI lead qualification analyst for a local home services business.  
  
Your job is to analyze a call transcript and extract structured lead  
information.  
  
Business:  
EliteFix Plumbing & HVAC  
  
Services:  
- AC repair  
- Plumbing repair  
- Water heater repair  
- Emergency maintenance
```

- Installation requests

Service Areas:

- Lahore
- DHA Lahore
- Gulberg
- Model Town
- Johar Town
- Bahria Town Lahore

Scoring Rules:

- Clear service need detected: 20 points
- Urgent timeline: 20 points
- Phone number provided: 15 points
- Name provided: 10 points
- Ready to book or requested callback: 15 points
- Location within service area: 10 points
- Budget or seriousness signal detected: 10 points

Lead Temperature:

- hot: 75 to 100
- warm: 45 to 74
- cold: 0 to 44

Return only valid JSON. Do not include markdown.

User prompt:

Analyze this call transcript and return structured JSON.

Transcript:

{{TRANSCRIPT}}

Required JSON format:

```
{
  "name": string | null,
  "phone": string | null,
  "email": string | null,
  "service_requested": string | null,
  "location": string | null,
  "urgency": string | null,
  "budget": string | null,
  "intent": string,
  "lead_score": number,
  "lead_temperature": "hot" | "warm" | "cold",
  "ai_summary": string,
```



```

    "recommended_action": string,
    "score_breakdown": {
      "clear_service_need": number,
      "urgent_timeline": number,
      "phone_number_provided": number,
      "name_provided": number,
      "ready_to_book": number,
      "in_service_area": number,
      "seriousness_signal": number
    }
  }
}

```

Important:

- The AI must not invent missing details.
- If a field is missing, return null.
- Score must be based only on the transcript.
- Total score must equal the sum of score_breakdown.
- lead_temperature must match the score.
- recommended_action should be practical and business-focused.

11. Lead Scoring Function

Create a deterministic helper function too.

File:

```
/lib/lead-scoring.ts
```

The function should calculate temperature from score:

```

export function getLeadTemperature(score: number): "hot" | "warm" | "cold" {
  if (score >= 75) return "hot";
  if (score >= 45) return "warm";
  return "cold";
}

```

Also validate score:

```

export function clampLeadScore(score: number): number {
  if (score < 0) return 0;
  if (score > 100) return 100;
}

```

```
    return score;  
}
```

12. UI Design Requirements

Use a modern SaaS dashboard design.

Style:

- Clean dark/light SaaS dashboard
- Sidebar navigation
- Card-based metrics
- Clear lead temperature badges
- Hot leads should visually stand out
- Use professional spacing and typography
- Mobile responsive

Sidebar links:

Dashboard
Leads
Demo Call
Notifications
Settings

Lead badges:

Hot = red badge
Warm = yellow/orange badge
Cold = gray badge

Status badges:

New
Contacted
Follow Up
Won
Lost

13. Seed Demo Data

Create seed data for at least 5 leads.

Lead 1 — Hot

```
{
  "name": "Ahmad",
  "phone": "0300-1234567",
  "service_requested": "AC repair",
  "location": "DHA Lahore",
  "urgency": "today",
  "lead_score": 92,
  "lead_temperature": "hot",
  "status": "new",
  "ai_summary": "Caller needs urgent AC repair today in DHA Lahore and is ready for a technician visit.",
  "recommended_action": "Call immediately and dispatch technician."
}
```

Lead 2 — Warm

```
{
  "name": "Sara",
  "phone": "0312-2223333",
  "service_requested": "Plumbing repair",
  "location": "Gulberg Lahore",
  "urgency": "this week",
  "lead_score": 63,
  "lead_temperature": "warm",
  "status": "follow_up",
  "ai_summary": "Caller is interested in plumbing repair but wants pricing before booking.",
  "recommended_action": "Send pricing details and follow up within 24 hours."
}
```

Lead 3 — Cold

```
{
  "name": "Ali",
  "phone": null,
  "service_requested": "AC installation",
  "location": null,
  "urgency": "not urgent",
}
```

```
"lead_score": 28,
"lead_temperature": "cold",
"status": "new",
"ai_summary": "Caller asked general questions about AC installation but did not provide contact details.",
"recommended_action": "No immediate action. Add to nurture list if contact details become available."
}
```

Lead 4 — Hot

```
{
  "name": "Usman",
  "phone": "0333-4445555",
  "service_requested": "Water heater repair",
  "location": "Model Town Lahore",
  "urgency": "emergency",
  "lead_score": 88,
  "lead_temperature": "hot",
  "status": "new",
  "ai_summary": "Caller needs emergency water heater repair in Model Town and requested immediate callback.",
  "recommended_action": "Call immediately and assign technician."
}
```

Lead 5 — Warm

```
{
  "name": "Mariam",
  "phone": "0321-9998888",
  "service_requested": "Bathroom plumbing",
  "location": "Johar Town Lahore",
  "urgency": "next few days",
  "lead_score": 70,
  "lead_temperature": "warm",
  "status": "new",
  "ai_summary": "Caller is looking for bathroom plumbing work in the next few days and provided contact details.",
  "recommended_action": "Call today and offer available appointment slots."
}
```

14. Notification System

Create a notifications page.

Route:

```
/notifications
```

Show:

- Notification type
- Lead name
- Channel
- Recipient
- Message preview
- Status
- Sent time

For MVP:

- Store notification in database.
- Show it in the UI.
- Add fake "Sent" status.

Later:

- Add email sending.
 - Add Slack webhook.
 - Add WhatsApp/SMS.
-

15. Human Handoff Rules

Create a function:

```
/lib/handoff.ts
```

Rules:

```
export function shouldTriggerHandoff(score: number): boolean {  
  return score >= 75;  
}
```

When handoff triggers:

1. Create notification.
2. Add lead event.
3. Show "Human handoff triggered" on lead detail page.
4. Add recommended action.

16. Environment Variables

Use these environment variables:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
  
OPENAI_API_KEY=  
  
APP_URL=http://localhost:3000  
  
HANDOFF_EMAIL=  
HANDOFF_PHONE=
```

For MVP, make sure the app works even if `OPENAI_API_KEY` is missing by using mock analysis mode.

17. Mock AI Analysis Mode

If no AI key is available, create a fallback function that analyzes transcript with simple keyword rules.

Example:

- If transcript includes "today", "urgent", "emergency", add urgency points.
- If transcript includes "AC", "plumbing", "water heater", detect service.
- If transcript includes phone number pattern, detect phone.
- If transcript includes "book", "send someone", "call me", mark ready to book.
- If transcript includes "DHA", "Gulberg", "Model Town", "Johar Town", mark in service area.

This lets the demo work without paid API usage.

18. Acceptance Criteria

The project is complete when:

1. Dashboard loads successfully.
 2. Demo leads are visible.
 3. User can open lead detail page.
 4. User can paste call transcript in Demo Call page.
 5. AI/mock analysis extracts lead details.
 6. Lead score is calculated.
 7. Lead temperature is assigned.
 8. Lead is saved to database.
 9. Hot lead creates a notification.
 10. User can update lead status.
 11. Settings page exists.
 12. App is clean, responsive, and demo-ready.
-

19. Suggested Folder Structure

```
voicelead-crm/  
  app/  
    page.tsx  
    leads/  
      page.tsx  
      [id]/  
        page.tsx  
    demo-call/  
      page.tsx  
    notifications/  
      page.tsx  
    settings/  
      page.tsx  
    api/  
      leads/  
        route.ts  
        [id]/  
          status/  
            route.ts  
      ai/  
        analyze-call/  
          route.ts  
      voice/  
        webhook/  
          route.ts  
      notifications/
```

```
    hot-lead/
      route.ts
components/
  dashboard/
    metric-card.tsx
    recent-leads.tsx
  leads/
    leads-table.tsx
    lead-temperature-badge.tsx
    lead-status-badge.tsx
    score-breakdown.tsx
  layout/
    sidebar.tsx
    topbar.tsx
lib/
  supabase/
    client.ts
    server.ts
  ai/
    analyze-call.ts
    mock-analyze-call.ts
  lead-scoring.ts
  handoff.ts
  notifications.ts
  types.ts
  utils.ts
supabase/
  schema.sql
  seed.sql
README.md
.env.example
```

20. Build Phases

Phase 1 — CRM UI

Build:

- Dashboard
- Leads page
- Lead detail page
- Seed demo data
- Static UI

Phase 2 — Database

Add:

- Supabase tables
- Create/read/update leads
- Lead events
- Notifications

Phase 3 — Demo Call Analyzer

Add:

- Demo Call page
- Transcript input
- Analyze call API
- Mock AI fallback
- Save analyzed lead

Phase 4 — Real AI

Add:

- OpenAI analysis function
- JSON validation with Zod
- Error handling
- Fallback to mock mode

Phase 5 — Human Handoff

Add:

- Hot lead threshold
- Notification creation
- Handoff event
- Hot lead alert UI

Phase 6 — Voice Integration Ready

Add:

- `/api/voice/webhook`
 - Provider field
 - Call ID field
 - Recording URL field
 - Transcript processing
-

21. README Requirements

Create a README that explains:

- What the project does
 - Demo use case
 - Features
 - Tech stack
 - Setup steps
 - Environment variables
 - How to run locally
 - How to test demo call
 - Future improvements
-

22. Future Improvements

After MVP, add:

- Real Vapi or Retell voice agent
 - Real phone number
 - Live call transfer
 - Google Calendar booking
 - WhatsApp alerts
 - Slack alerts
 - Email follow-up
 - Multi-business support
 - Team member accounts
 - Role-based access
 - Call recording playback
 - Analytics dashboard
 - Lead conversion tracking
 - CRM integrations: HubSpot, Pipedrive, GoHighLevel
-

23. Final Product Goal

The final demo should clearly show this:

A customer calls a business. The AI answers, qualifies the caller, scores the lead, saves everything in CRM, and alerts the human team when the caller is ready to buy.

This should look like a real client-ready AI automation system, not a basic chatbot.